

PORTFOLIO + GUÍA DE ESTUDIO

Next.js & Tailwind CSS

De lo más básico a lo más técnico, con comparativa con Angular
y preguntas frecuentes de entrevistas

Documento de aprendizaje para el portafolio personal
Autor: Desarrollador Junior Full Stack (Spring Boot + Angular + React/Next)
Estructura: 5 partes + cheatsheet de comandos

Generado con Python + ReportLab

Índice

Índice	2
1. Next.js: de lo básico a lo técnico	3
1.1 ¿Qué es Next.js?	3
1.2 Requisitos y primeros pasos	3
1.3 Estructura de carpetas (App Router)	3
1.4 Componentes: Server vs Client	3
1.5 Tipos de renderizado	4
1.6 Obtención de datos (data fetching)	4
1.7 SEO y Metadata API	5
1.8 next/image y next/font (optimización incluida)	5
1.9 Fuentes auto-hospedadas (self-hosted)	5
1.10 Despliegue	5
2. Tailwind CSS: de lo básico a lo técnico	6
2.1 ¿Qué es Tailwind CSS?	6
2.2 Instalación en Next.js (Tailwind v4)	6
2.3 Sistema de tamaño: escala de 4px (0.25rem)	6
2.4 Colores	6
2.5 Responsive con breakpoints	6
2.6 Estados: hover, focus, activo	6
2.7 Modo oscuro	6
2.8 Reutilización: componentes antes que @apply	7
2.9 Personalización con @theme (v4)	7
2.10 Ejemplo completo: una card	7
3. Comparativa: Angular vs Next.js	7
3.1 Respuesta directa a tu duda sobre el CLI	7
3.2 Tabla de equivalencias CLI	8
3.3 Conceptos equivalentes	8
3.4 Dificultades típicas al migrar la mentalidad de Angular	8
4. Preguntas frecuentes de entrevistas sobre Next.js	9
5. Cheatsheet: comandos y snippets rápidos	11
5.1 Comandos	11
5.2 Snippet: Server Action básico	11
5.3 Snippet: ruta dinámica con params	11
5.4 Snippet: estático con incremento (ISR)	11

1. Next.js: de lo básico a lo técnico

1.1 ¿Qué es Next.js?

Next.js es un **framework de React** creado y mantenido por Vercel. Sobre React añade un conjunto de decisiones *opiniadas* para que una aplicación no la armes pieza a pieza: **enrutado**, **renderizado en servidor**, **optimización de imágenes**, **SEO**, **API interna** (rutas de servidor) y **despliegue** listo para producción.

Se le llama **framework full-stack** porque con un solo proyecto puedes crear tanto la interfaz (React) como el servidor (API route handlers, server actions), igual que tu proyecto de Spring Boot + Angular, pero con React en lugar de Angular y Node en lugar de Java.

1.2 Requisitos y primeros pasos

```
# Requisitos mínimos: Node.js 18.18+ y npm (es una app de JavaScript, no necesitas instalar nada más)
node --version # verifica tu versión de Node

# Crear un proyecto nuevo (equivalente a "ng new")
npx create-next-app@latest mi-portafolio

# Responde las preguntas: TypeScript, ESLint, Tailwind, App Router, src dir
# Comandos que usas todos los días:
npm run dev      # servidor de desarrollo (equivalente a "ng serve") -> http://localhost:3000
npm run build    # build de producción (equivalente a "ng build")
npm run start    # sirve el build de producción
npm run lint     # ejecuta ESLint (equivalente a "ng lint")
```

1.3 Estructura de carpetas (App Router)

A diferencia de Angular (con src/app/modules, services, etc.), Next.js fija las **convenciones de archivos**. La carpeta src/app/ define las rutas del sitio: cada page.tsx es una página.

```
mi-portafolio/
├── public/           # archivos estáticos (logo, favicon, imágenes)
├── src/
│   └── app/
│       ├── layout.tsx      # layout raíz: envuelve TODAS las páginas
│       ├── page.tsx        # la ruta "/"
│       ├── globals.css     # estilos globales (aquí se importa Tailwind)
│       ├── about/page.tsx  # la ruta "/about"
│       └── api/route.ts    # una ruta de API (Route Handler)
│   ├── components/        # tus componentes de React (Navbar, Hero, ProjectCard...)
│   ├── data/              # datos que centralizas (projects.ts, site.ts)
│   └── lib/               # lógica de negocio: "servicios" (funciones y helpers)
├── package.json
├── next.config.ts
└── tsconfig.json
```

No hay un CLI que genere estos archivos (como ng g component). El 'generador' de Next.js es la **estructura de carpetas y archivos con nombres convencionales**: page.tsx, layout.tsx, route.ts, loading.tsx, error.tsx, not-found.tsx.

1.4 Componentes: Server vs Client

Es la idea más importante del App Router. Por defecto **todo es Server Component** (se ejecuta en el servidor y no envía JS al navegador). Solo cuando necesitas interactividad (estado, eventos, hooks del navegador) marcas el archivo con 'use client' al inicio.

```
// Server Component (por defecto): corre en el servidor
// Sirve para: leer datos de API/BD, importar librerías pesadas, SEO, sin JS en el cliente.
async function Usuarios() {
  const res = await fetch("https://api.ejemplo.com/users"); // fetch directo en el servidor
  const users = await res.json();
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

// Client Component: con "use client" al inicio del archivo
'use client';
import { useState } from "react";

export default function Contador() {
  const [n, setN] = useState(0); // estado solo funciona en el cliente
  return <button onClick={() => setN(n + 1)}>{n}</button>;
}
```

1.5 Tipos de renderizado

Comparado con Angular (que por defecto es una SPA / CSR), Next.js ofrece 4 estrategias de rendering:

Estrategia	Qué es	Cuándo se genera	Uso típico
CSR (Client Side Rendering)	React renderiza en el navegador (como Angular SPA)	En el cliente	Dashboards, apps con estado muy dinámico, requiere autenticación
SSG (Static Site Generation)	La página se genera una vez en build y se sirve como HTML estático	En build	Landings, blogs, documentos (tu portafolio)
SSR (Server Side Rendering)	La página se genera por cada petición en el servidor	En cada request	Datos personalizados que cambian seguido
ISR (Incremental Static Regeneration)	SSG pero con revalidación automática	En build + revalida cada X segundos	Catálogo, precios, noticias (dato que cambia a menudo)

En el App Router NO eliges una de ellas de forma explícita: Next.js lo decide de forma inteligente. Por defecto intenta renderizar **estático** (SSG). Puedes usar `export const dynamic = "force-static"` o `revalidate` para forzar SSR/ISR.

```
// SSG estático (default en App Router): no es necesario configurar nada
export const revalidate = 60; // ISR: revalida la página cada 60 segundos
// o bien:
export const dynamic = "force-static"; // SSG siempre
export const dynamic = "force-dynamic"; // SSR en cada request

// Tu portafolio NO necesita ninguna de las dos: es todo estático por defecto.
```

1.6 Obtención de datos (data fetching)

Ya no hay `HttpClient` ni `RxJS`. En el servidor puedes usar `fetch()` nativo directamente en un Server Component con `async/await`.

```
// Data fetching SIN dependencias (la forma moderna en Next.js):
async function getProyectos() {
  const res = await fetch("https://api.tuserver.com/proyectos", { cache: "no-store" });
  if (!res.ok) throw new Error("No se pudo obtener los proyectos");
  return res.json();
}

export default async function Proyectos() {
  const proyectos = await getProyectos();
  return <section>{proyectos.map(proj => <Card key={proj.id} titulo={proj.title} />)}</section>;
}
```

Route Handlers son tu 'API' dentro de Next.js (equivalente a tus `@RestController` de Spring Boot, pero como capa de redirección, no reemplazan un backend real):

```
// src/app/api/saludo/route.ts -> expone POST /api/saludo
import { NextResponse } from "next/server";

export async function POST(req: Request) {
  const body = await req.json(); // lee el cuerpo de la petición
  const nombre = body.nombre ?? "mundo";
  return NextResponse.json({ mensaje: `Hola ${nombre}` });
}
```

1.7 SEO y Metadata API

Con `export const metadata: Metadata = {...}` en cualquier `layout.tsx` o `page.tsx`, Next.js genera `title`, `meta description`, Open Graph, etc. Además puedes generar `sitemap.xml` y `robots.ts` sin tocar HTML.

```
// En tu layout.tsx (equivala al index.html + meta tags de una app Angular):
import type { Metadata } from "next";

export const metadata: Metadata = {
  title: "Mi Portafolio - Dev Junior",
  description: "Portafolio con Spring Boot, Angular y proyectos personales.",
  openGraph: { title: "Mi Portafolio", type: "website" },
};
```

1.8 next/image y next/font (optimización incluida)

`next/image` optimiza imágenes: las convierte a WebP/AVIF, añade lazy loading automático y exige **width y height** (o `fill`) para evitar layout shift. `next/font` sirve tipografías auto-hospedadas **sin peticiones externas**, mejorando velocidad y privacidad.

```
import Image from "next/image";

// Obligatorio pasar width/height (o fill dentro de un contenedor posicionado)
<Image src="/foto.jpg" alt="Foto de perfil" width={400} height={400} className="rounded-full" priority />
```

1.9 Fuentes auto-hospedadas (self-hosted)

En este portafolio usamos `next/font/local` con las fuentes Geist y Geist Mono guardadas en `src/fonts/`. Así el build no depende de Google Fonts (puede estar bloqueada en tu red o en la de producción).

```
import localFont from "next/font/local";

const geistSans = localFont({ src: "../fonts/Geist-Variable.woff2", variable: "--font-geist-sans" });
const geistMono = localFont({ src: "../fonts/GeistMono-Variable.woff2", variable: "--font-geist-mono" });
```

1.10 Despliegue

La forma más fácil es **Vercel** (la empresa de Next.js): conectas tu repo de GitHub y cada `git push` genera un deploy con HTTPS y dominio gratis. También puedes desplegar en cualquier hosting que corra Node, o exportar HTML estático con `output: 'export'` en `next.config.ts` (así el sitio se puede subir a GitHub Pages o Netlify).

2. Tailwind CSS: de lo básico a lo técnico

2.1 ¿Qué es Tailwind CSS?

Tailwind es un framework CSS **utility-first**: en lugar de escribir clases semánticas en un archivo CSS (como en Angular: `.btn-alerta{...}`), compones utilidades directamente en el `class` de tus etiquetas: `bg-red-500`, `px-4`, `rounded-lg`, `md:hidden`. Eso evita inventar nombres de clase y mantiene el CSS casi sin escribir.

2.2 Instalación en Next.js (Tailwind v4)

En esta app (Next.js 16 + Tailwind v4) la configuración ya NO es un archivo `tailwind.config.ts` como en v3: ahora Tailwind se configura dentro del CSS. El único archivo necesario es `globals.css`:

```
/* src/app/globals.css */
@import "tailwindcss";          /* 1) trae todo el framework */

@theme {                        /* 2) define tokens: colores, fuentes personalizadas */
  --color-accent: #10b981;
  --font-sans: var(--font-geist-sans);
}
/* 3) ahora puedes usar: bg-accent, text-accent, font-sans... */
```

2.3 Sistema de tamaño: escala de 4px (0.25rem)

Las utilidades de espaciado y tamaño usan una escala incrementada de **0.25rem (4px)**: `p-1` = 4px, `p-4` = 16px, `p-10` = 40px. Casi nunca escribes valores sueltos: usas `p-0` ... `p-96`, y además `px-` (horizontal), `py-` (vertical), `mx-/my-` (márgenes).

2.4 Colores

Los colores siguen el patrón `color-escala`. La escala va de 50 (más claro) a 950 (más oscuro).

```
text-zinc-500      # texto gris medio (tono "zinc" de Tailwind)
bg-accent          # usa tu token personalizado definido en @theme
border-border      # token :root para los bordes
bg-surface/50      # el sufijo "/50" (opacity) no existe en Angular: 50 = 50% de transparencia
```

2.5 Responsive con breakpoints

Tailwind es **mobile-first**: las clases base aplican al móvil y prefijas breakpoints para pantallas mayores. No escribes media queries a mano.

```
<div class="grid-cols-1 md:grid-cols-2 lg:grid-cols-3">
  <!-- móvil: 1 columna | tablet (>=768px): 2 | desktop (>=1024px): 3 -->
</div>
sm = >=640px   md = >=768px   lg = >=1024px   xl = >=1280px   2xl = >=1536px
```

2.6 Estados: hover, focus, activo

```
<button class="bg-accent hover:bg-accent-light focus:outline-2 active:scale-95">
  Enviar
</button>
```

2.7 Modo oscuro

En Tailwind v4 controlas el modo claro/oscuro con el modificador `dark`: activado por una clase `.dark` sobre `html`. Este portafolio es oscuro de base: define los colores ('tokens') y usa utilitarios concretos.

2.8 Reutilización: componentes antes que @apply

La filosofía de Tailwind dice: **no abras un archivo CSS**. Si repites utilidades, crea un **componente de React** y centralízalo. En Next.js eso es un archivo nuevo en `src/components/` (lo que hace tu portafolio con `ProjectCard`). Si aún así quieres reutilizar en CSS (muy puntual), Tailwind v4 permite usar la directiva `@apply`:

```
/* Solo para casos muy puntuales: NO abusos de @apply */
.card {
  @apply rounded-xl border border-border bg-surface p-6;
}
```

2.9 Personalización con @theme (v4)

En v4 defines tus *design tokens* en `@theme` y Tailwind genera automáticamente las utilidades. Este portafolio define `--color-accent`, `--color-surface`, `--color-border` y `--font-sans`.

2.10 Ejemplo completo: una card

```
<article class="rounded-xl border border-border bg-surface p-6 hover:border-accent/60">
  <h3 class="text-lg font-semibold text-zinc-50">Título</h3>
  <p class="mt-2 text-sm text-zinc-400">Descripción</p>
  <span class="mt-3 inline-block rounded bg-border/60 px-2 py-0.5 font-mono text-[11px] text-zinc-300">
    Java
  </span>
</article>
```

3. Comparativa: Angular vs Next.js

3.1 Respuesta directa a tu duda sobre el CLI

En Angular usas `ng g service nombre --skip-tests` para generar archivos. **En Next.js NO existe un CLI generador como ese**. Los archivos se crean a mano siguiendo convenciones de nombres; el 'framework' de Next es el sistema de carpetas y archivos. Para un 'servicio' (lógica reutilizable) creas una función en `src/lib/` y la importas:

```
// "Servicio" equivalente en Next.js (sin CLI, sin decoradores @Injectable):
// src/lib/greeting.ts
export function saludar(nombre: string) {
  return `Hola ${nombre}!`;
}

// uso en cualquier página o componente:
import { saludar } from "@lib/greeting";
<p>{saludar("Ana")}</p>
```

3.2 Tabla de equivalencias CLI

Acción	Angular CLI	Next.js / npm
Crear proyecto	ng new mi-app	npx create-next-app@latest mi-app
Servir en desarrollo	ng serve -o	npm run dev
Generar componente	ng g c componentes/nombre	Manual: crear components/Nombre.tsx (no hay CLI de generación)
Generar servicio	ng g service servicio --skip-tests	Manual: crear lib/servicio.ts con una función
Generar módulo	ng g m modulo	No existe: la estructura de src/app/ es tu 'módulo'
Guard / rutas	ng g guard ... + app-routing	Manual + middleware.ts, o carpeta + page.tsx
Pipe / interfaz	ng g pipe / ng g interface	Manual: función normal / type en TS
Build de producción	ng build	npm run build
Lint	ng lint	npm run lint (ESLint ya configurado)
Tests	ng test / ng e2e	npm test (configuras Vitest/Jest) - no viene por defecto
Añadir librería	ng add <paquete>	npm install <paquete> + config manual
UI Components	ng generate + Angular Material	npx shadcn add button (ecosistema shadcn/ui)

3.3 Conceptos equivalentes

Concepto Angular	Equivalente en Next.js / React
AppModule + declarations	layout.tsx + componentes importados
Routing (app-routing.module.ts)	Carpetas + page.tsx (file-based routing)
Ruta con parámetro (id)	app/usuario/[id]/page.tsx; recibe params: { id }
ActivatedRoute.paramMap	En server: params de la página · en client: useParams()
@Injectable + inyección de dependencias	Importar funciones/constantes (no hay DI: proximidad por imports)
HttpClient + interceptores	fetch() nativo o axios; a nivel HTTP usa middleware.ts
RxJS + Observables	async/await + fetch; estado reactivo con hooks o librerías
Guards de rutas	middleware.ts (se ejecuta antes de llegar a una página)
ngIf / ngFor / ngSwitch	Condicionales ternarios + .map() en JSX
Pipes (date, uppercase)	Funciones de JS (new Date().toLocaleDateString('es-ES'))
@Input() / @Output()	props (de padre a hijo) / callbacks (onClick, onX)
ReactiveFormsModule + validators	react-hook-form + zod (librerías de React)
ngOnInit / ciclo de vida	useEffect (solo en client components)
ChangeDetectionStrategy	React re-renderiza automáticamente con estado (no lo configuras)
Servicios singleton	Import global del módulo ES + const compartida

3.4 Dificultades típicas al migrar la mentalidad de Angular

- **No hay 'módulos':** no necesitas declarar dependencias. Si importas algo, simplemente funciona.
- **Los componentes son por defecto server-side:** un useEffect o un onClick en un Server Component da error; añade 'use client' arriba de ese archivo.

- **La reactividad es con estado:** para que una variable cambie la UI, usa hooks (useState, useMemo).
- **No hay inyección automática:** importas cada dependencia de forma explícita (puedes crear singletons con ES Modules).
- **No manipulas el DOM:** no hay *ngFor ni @ViewChild; renderizas de nuevo con .map().
- **Rutas dinámicas:** creas carpetas con corchetes ([id]) en lugar de definir un array de rutas.

4. Preguntas frecuentes de entrevistas sobre Next.js

Respuestas cortas que demuestran dominio. Repásalas 10 minutos antes de una entrevista. Normalmente preguntan 'cuándo usarías cada una': piensa siempre en tu proyecto real (portafolio, sistema de pedidos).

1. ¿Qué es Next.js y por qué usarlo en vez de React puro?

Next.js es un framework de React que añade convenciones: enrutado por archivos, renderizado en servidor (SSR/SSG/ISR), optimización de imágenes, SEO y API interna. React puro solo pinta en el navegador; Next.js te da una SPA con toda la capa de servidor sin configurarla tú.

2. ¿Qué es el App Router? ¿Cómo se diferencia del Pages Router (v12 y anteriores)?

El App Router (13.4+) usa la carpeta app/ con convenciones de archivos (page, layout, loading, error, route) y trae Server Components, layouts anidados y streaming. El Pages Router (carpeta pages/) era la API anterior: cada archivo era una página y todo era Client Component.

3. ¿Qué son los Server Components y los Client Components?

Los Server Components corren en el servidor, no envían JS al navegador (ideales para datos y SEO) y son el comportamiento por defecto. Los Client Components se marcan con 'use client' al inicio del archivo y son los únicos que pueden usar hooks (useState, useEffect) y eventos. Lleva al cliente solo lo necesario.

4. Explica SSR, SSG, ISR y CSR. ¿Cuándo usarías cada una?

SSG genera HTML en build (ideal para el portafolio). SSR renderiza en cada request (datos personalizados). ISR es SSG que se revalida cada T segundos (precios/noticias). CSR renderiza todo en el navegador (dashboards con estado vivo). En el App Router la decisión es automática: por defecto intenta ser estático; con revalidate/dynamic la fuerzas.

5. ¿Qué significa 'use client'?

Es una directiva (no un import) que indica que ese archivo (y lo que importe) se ejecutará en el navegador. Se coloca como primera línea del archivo.

6. ¿Cómo haces fetching de datos en un Server Component?

Con fetch() nativo en una función async y await. Puedes cachear con 'cache: 'force-cache"' (por defecto) u omitir la caché con 'cache: 'no-store'".

7. ¿Qué es un Route Handler y cuándo lo usarías?

Un archivo route.ts dentro de app/ que exporta GET, POST, PUT, DELETE... Expone un endpoint HTTP en tu app. Sirve como capa intermedia, webhooks o para no exponer credenciales en el cliente (el código corre en el servidor).

8. ¿Qué es un Server Action?

Una función async marcada con 'use server' que se ejecuta en el servidor y puede llamarse desde un formulario o cliente (mutation). Es la evolución de los formularios de Next.js: evita montar un endpoint API para una mutación.

9. ¿Cómo haces SEO en el App Router?

Exportando metadata desde layout.tsx o page.tsx (Metadata API): title, description, openGraph, robots, etc. Next.js genera las etiquetas del head. También tienes sitemap.ts y robots.ts.

10. ¿Para qué sirve next/image?

Optimiza imágenes automáticamente: lazy loading, resize, formato WebP/AVIF y previene layout shift al exigir width/height (o fill). Funciona con imágenes locales y remotas (configurando remotePatterns).

11. ¿Qué es el middleware.ts de Next.js?

Código que corre antes de las rutas (equivale a un guard o interceptor de Angular) para validar sesión, redirigir o modificar la request. Se ejecuta en el edge.

12. ¿Cómo optimizarías el rendimiento de una página Next.js?

Priorizando Server Components, usando next/image y next/font, carga diferida con next/dynamic, reduciendo JS en el cliente, minimizando re-renders (useMemo/useCallback solo cuando hacen falta) y cacheando consultas (ISR, revalidate).

13. ¿Para qué sirve next/dynamic?

Carga un componente bajo demanda (code splitting dinámico), como en 'lazy' de bibliotecas pesadas que no hacen falta al inicio (gráficas, editores).

14. ¿Qué son loading.tsx, error.tsx y not-found.tsx?

Son 'boundaries' opcionales del App Router: loading.tsx muestra un estado de carga (suspense), error.tsx captura errores de su sección y not-found.tsx personaliza el 404.

15. ¿Cómo desplegarías este portafolio gratis?

Con Vercel conectado a GitHub: deploy automático con cada push, HTTPS y dominio gratis. Alternativa: exportar estático (output: 'export') y subirlo a GitHub Pages o Netlify.

16. ¿Cuándo usas un Server Component vs un Client Component?

Servidor: leer BD/API, sesión, SEO, logs, librerías pesadas. Cliente: formularios interactivos, drag & drop, estado, animaciones que dependen del DOM. Regla práctica: empieza en servidor y baja al cliente solo lo imprescindible.

17. ¿Qué diferencia hay entre una SPA de Angular y una app Next.js (App Router)?

Una SPA de Angular descarga JS y renderiza todo en el navegador. Next.js decide en el servidor: la primera respuesta trae HTML listo (SEO/velocidad) y crece de forma incremental. Ambos usan TypeScript de serie.

18. ¿Cómo manejas variables de entorno de forma segura?

Con archivos .env.local y .env. Las variables públicas van con el prefijo NEXT_PUBLIC_ y se exponen en el cliente. Las privadas (claves, tokens) se leen solo en código de servidor, nunca con NEXT_PUBLIC_. Nunca subas secretos a GitHub.

5. Cheatsheet: comandos y snippets rápidos

5.1 Comandos

Comando	Qué hace
<code>npx create-next-app@latest mi-app</code>	crear proyecto
<code>npm run dev</code>	servidor de desarrollo
<code>npm run build</code>	build de producción
<code>npm run start</code>	servir build
<code>npm run lint</code>	ESLint
<code>npx next dev --turbo</code>	dev con Turbopack (más rápido)
<code>npx shadcn@latest init / add</code>	componentes UI generados

5.2 Snippet: Server Action básico

```
// En el archivo donde quieras la acción o en un archivo "use server":
"use server";

export async function crearProyecto(formData: FormData) {
  const titulo = formData.get("titulo");
  // ... inserta, procesa, redirige
  return { ok: true };
}
```

5.3 Snippet: ruta dinámica con params

```
// app/proyecto/[id]/page.tsx
export default async function ProyectoPage({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  const proyecto = await getProyecto(id);
  return <main>{proyecto.title}</main>;
}
```

5.4 Snippet: estático con incremento (ISR)

```
// En un page.tsx o route handler:
export const revalidate = 3600; // se regenera como máximo cada 1h
```

Fin de la guía. Siguiendo el siguiente paso recomendado: abre el código de tu portafolio en `src/app/layout.tsx`, `src/data/*.ts` y los componentes de `src/components/`, y repasa los comentarios: están pensados como mini-lecciones.